

Machine Learning in Python

Lab: Decision Trees (ID3 and C4.5)

Gdańsk University of Technology

Introduction

Decision trees are intuitive, interpretable models used for classification tasks. The classic ID3 (Iterative Dichotomiser 3) algorithm relies on Shannon's Entropy and Information Gain to split categorical data. However, ID3 is inherently limited: it struggles with continuous (numerical) attributes, often leading to deep, overfitted trees where each unique numerical value is treated as a distinct category.

The C4.5 algorithm introduces significant improvements. It utilizes the Gain Ratio to penalize attributes with many distinct values and can dynamically determine optimal split thresholds for continuous data. Furthermore, C4.5 implements post-pruning techniques, such as Pessimistic Error Pruning (PEP), to reduce tree complexity and mitigate overfitting.

Task 0: Base ID3 on Categorical Data

First, evaluate the standard ID3 algorithm using a purely discrete dataset. The `spacer.csv` file contains only categorical labels for attributes like Outlook, Wind, and Humidity. Use the provided execution pipeline to run the ID3 script and examine the generated tree structure.

```
python 01_id3.py --data data/spacer.csv --target Spacer
```

Programming Implementation Note:

[PLACEHOLDER: Describe the core logic, functions, and data structures required to program the base ID3 algorithm for categorical features here.]

To pass the task: Generate the decision tree graph in the `out/` directory. Verify that the algorithm splits the nodes exclusively on exact categorical matches, note the depth of the resulting tree, and save the generated `.png` image for comparison.

Task 1: ID3 Overfitting on Continuous Data

Next, evaluate how the base ID3 algorithm handles continuous numerical values. Use the `spacer_liczbowy.csv` dataset, where the `Humidity` column is defined by continuous integers rather than discrete text labels.

```
python 01_id3.py --data data/spacer_liczbowy.csv --target Spacer
```

Programming Implementation Note:

[PLACEHOLDER: Explain the programmatic limitations of the ID3 feature selection logic when applied to numerical arrays or continuous Pandas Series.]

To pass the task: Generate the graph and observe the structural branching for the `Humidity` attribute. Document how ID3 treats each unique integer as a separate standalone category, leading to an overly wide, overfitted tree branch, and compare this visual output with the categorical tree from Task 0.

Task 2: C4.5 Continuous Split Handling

The C4.5 algorithm natively processes continuous variables by sorting the numeric values and calculating the Gain Ratio for potential binary splits (e.g., `> threshold` and `<= threshold`). Run the upgraded algorithm on the numerical dataset to observe this mechanism.

```
python 02_c45.py --data data/spacer_liczbowy.csv --target Spacer
```

Programming Implementation Note:

[PLACEHOLDER: Detail the step-by-step programming strategy to compute Gain Ratio, evaluate split info, and locate optimal threshold candidates in Python.]

To pass the task: Execute the script and identify the exact numerical threshold C4.5 selected to split the `Humidity` attribute. Briefly explain why a binary continuous split yields a higher Gain Ratio than the multi-way categorical split seen in Task 1.

Task 3: Pessimistic Error Pruning

Deep decision trees fit the training data perfectly but often generalize poorly to unseen data. C4.5 utilizes Pessimistic Error Pruning (PEP) to cut back branches that do not provide statistically significant predictive power, calculating an upper-bound error rate for each sub-tree.

Programming Implementation Note:

[PLACEHOLDER: Outline the recursive tree-traversal logic and formula implementation required to code the Pessimistic Error Pruning (PEP) function.]

To pass the task: Review the console output and the two distinct visualizations generated by the C4.5 script in the `out/` directory: `c45_raw_tree.png` and `c45_pruned_tree.png`. Identify which branches were collapsed into leaf nodes by the PEP algorithm and record the difference in total node count before and after the pruning phase.